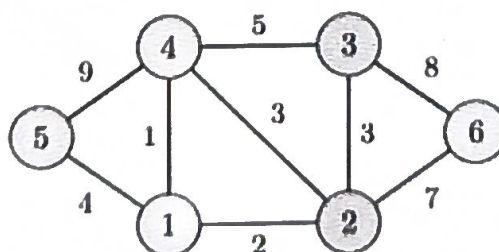


Course No: CSE 2202 || Lab Final || Total Marks: 15 || Time: 1 hours

- Q1. A large metropolitan hospital receives thousands of patients each day through its Emergency Department triage desks. Each patient check-in is recorded with a unique patient ID and an exact UNIX timestamp indicating the moment they entered the queue. These records are transmitted to the hospital server from multiple reception desks in no particular order, resulting in a disorganized stream of entries. Before the system can allocate doctors and prioritize treatment, it must arrange all check-in records in increasing order of arrival time so that no patient is seen out of sequence. Because new records may continuously arrive and the total number of patients can grow into the millions, the hospital's IT policy mandates the use of a comparison-based sorting algorithm that follows a divide-and-conquer strategy to reliably organize the data regardless of the order in which it arrives. Furthermore, since the data may not fully fit in cache at once, the sorting method must be capable of using auxiliary memory during the merging or combination phase in order to maintain efficient and stable performance. The system should also preserve the original order of patients who arrived at the exact same time, ensuring fairness in handling ties. Your task is to design and implement this sorting solution to process raw check-in data and output the sorted list of patients according to their arrival timestamps. 05

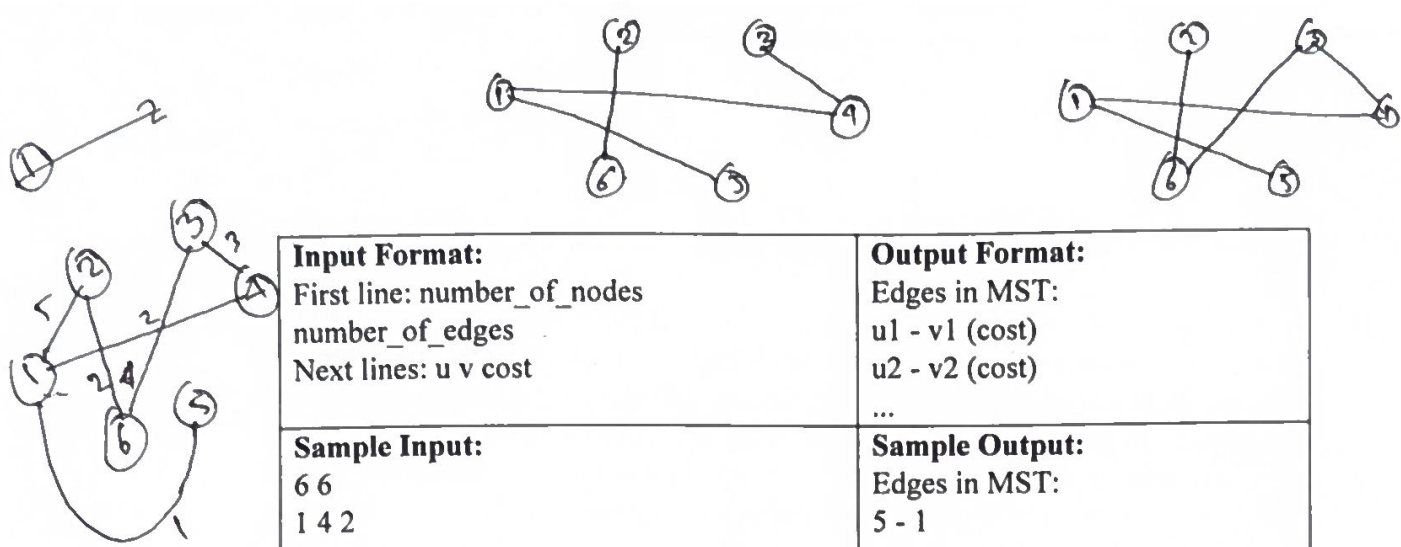
Input Format: N $patient_id_1\ checkin_time_1$ $patient_id_2\ checkin_time_2$... $patient_id_N\ checkin_time_N$	Output Format: $patient_id\ checkin_time$
Sample Input: 5 P301 1719931200 P115 1719925000 P890 1719931200 P441 1719928000 P552 1719940000	Sample Output: P115 1719925000 P441 1719928000 P301 1719931200 P890 1719931200 P552 1719940000
Constraints: $1 \leq N \leq 10^6$ $0 \leq checkin_time \leq 10^{12}$ - Built-in sorting functions (like <code>sort()</code> or <code>sorted()</code>) are not allowed.	

- Q2. A university campus is installing high-speed emergency fiber lines between buildings. Each connection has an associated installation cost. To minimize budget while ensuring all buildings are connected, the network team must construct a Minimum Spanning Tree (MST) — connecting every building with minimum total cable cost and no cycles. The map below shows buildings and possible fiber routes with their respective costs. 05



Edges $\rightarrow$ min
if u and v is visited
don't take it as it will
be a cycle

~ u v



Input Format: First line: number_of_nodes number_of_edges Next lines: u v cost	Output Format: Edges in MST: u1 - v1 (cost) u2 - v2 (cost) ...
Sample Input: 6 6 1 4 2 4 3 3 3 6 4 6 2 2 2 1 5 5 1 1	Sample Output: Edges in MST: 5 - 1 1 - 4 6 - 2 4 - 3 3 - 6

Q3. A bioinformatics research center is analyzing the evolution of a virus by comparing DNA samples from two different patients. Each DNA sequence consists of nucleotides represented by the characters A, C, G, T. To measure how closely the viral strains are related, the scientists must compute the Longest Common Subsequence (LCS) — the longest sequence of nucleotides that appears in the same order (not necessarily contiguously) in both samples. In addition to knowing the length of this longest subsequence, the researchers also need to reconstruct and print the subsequence itself, as it will be further studied to identify mutation hotspots. Your task is to determine the length of the LCS and the subsequence.

Input Format: X Y Where, • X = first DNA sequence • Y = second DNA sequence	Output Format: Length of LCS LCS string
Sample Input: ACGTACGT AACCTGTA	Sample Output: 5 ACGTA
Constraints: • $1 \leq X , Y \leq 1000$ • Characters $\in \{A, C, G, T\}$	

0 1 2 3
 ACCT
 TCTA

	0	1	2	3
0	0	0	0	1
1	0	1	1	1
2	0	1	2	1
3	1	1	2	2

CT
 TC

[152 155] 153 154 152